



Unit #: 2

Unit Name: Counting , Sorting and Searching

Topic: Callback

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and its associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and its respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Introduction

In computer programming, a **callback** is also known as a "**call-after function**". The callback execution may be immediate or it might happen at a later point in time. Programming languages support call backs in different ways, often implementing them with subroutines, lambda expressions, blocks, or function pointers. In C, a **callback function is a function that is called through a function pointer/pointer to a function**. A callback function has a specific action which is bound to a specific circumstance. It is an important element of GUI in C programming.

Let us understand the use of **pointer to a function or a function pointer**.

Function Pointer points to code, not data. The function pointer stores the start of executable code. The function pointers points to functions and store its address.

Syntax: `int (*ptrFunc) ();`

The **ptrFunc** is a pointer to a function that takes no arguments and returns an integer. If parentheses are not given around a function pointer then the compiler will assume that the ptrFunc is a normal function name, which takes nothing and returns a pointer to an integer. Function pointers are not used for allocating or deallocating memory, instead used in reducing code redundancy.

Consider the below cases:

Case 1: `int *a1(int, int, int);` // a1 is a function which takes three int arguments and returns a pointer to int.

Case 2: `int (*p)(int, int, int);` //p is a pointer to a function which takes three int as parameters and returns an int.

We can assign a function name to p as long as the function has the same signature which means the function takes three integer parameters and returns an integer. The function name acts like a pointer to a function.

```
int a2(int,int,int);  
p = a2;  
int y = p(2,3,4); // This statement is the same as calling a2.
```

Why is the callback function required?

Let us answer a few questions to understand the importance of callback function.

How to write a generic library for sorting algorithms such as bubble sort, shell sort, shake sort, quick sort?

How to create a notification event to set a timer in your application?

How to have one common method to develop libraries and event handlers for many programming languages?

How redirect page action is performed for the user based on one click action?

How to extend the features of one function using another one?

A callback is any executable code that is passed as an argument to other code, which is used to call (execute) the argument at a given time. In simple language, if a function name is passed to another function as an argument to call it, then it will be called as a Callback function.

Coding Example_1: Consider the below simple example to understand callback

```
int what(int x, int y, int z, int (*op)(int, int,int))  
{  
    return op(x, y, z);  
}
```

Observe the third argument op. It is a pointer to a function which takes three int arguments and returns an int. The return statement in turn calls the function with three arguments. This function does not know which function is getting called. It depends on the value of op. Compiler knows the type and not the value. The value of a variable is a runtime mechanism.

```
#include<stdio.h>

int what(int x, int y, int z, int (*op)(int, int,int))
{
    return op(x, y, z);
}

int add(int x, int y, int z)
{
    return x + y + z;
}

int mul(int x, int y, int z)
{
    return x * y * z;
}

int main() {
    int (*p)(int, int, int);

    p = add; // p = &add; Both are equivalent
    int res = p(2,3,4); // (*p)(2, 3, 4); //Both are equivalent
    printf("result is %d\n", res);

    p = mul;
    res = p(2,3,4);
    printf("result is %d\n", res);
    // function name can be directly passed as argument to function with callback
    printf("sum is %d\n", what(1, 3, 4, add));
    printf("product is %d\n", what(5, 3, 4, mul));
    return 0;
}
```

Happy Calling Back!!